

Code Smell과 리팩토링

코드스멜과 분류

코드 스멜의 패턴을 파악해보고, 이를 해결하는 리팩토링 기법들을 알아본다.



Chapter1

불필요한 복잡성 Smell

Code Smell의 분류

불필요한 복잡성을 띄는 Code Smell

SW 의 복잡도는 개발자들의 작업시간에 영향을 끼친다.

따라서 불필요한 복잡성을 제거하여 가독성이 좋고, 이해하기 좋은 구조로 변경해야 한다.

Code Smell 패턴의 종류

- 긴 함수 (Long Method)
- 거대한 클래스 (Large Class)
- 추측성 일반화 (Speculative Generality)

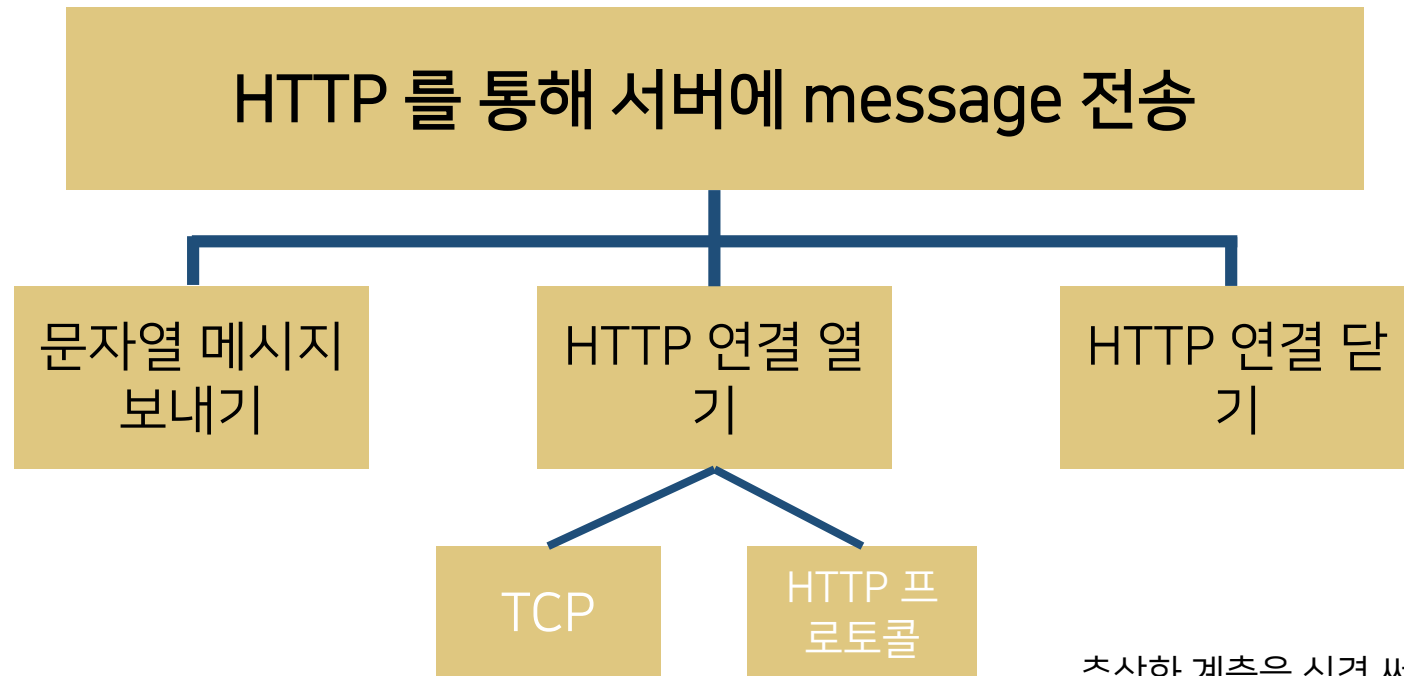
[Bad Smell] Long Method (긴 함수)

- 긴 함수는 이해하기 어렵다
코드가 길어지면 이해하기 어려워진다.
- 긴 함수는 여러가지 동작을 하고 있을 수 있다
코드는 한 가지를 잘해야 한다.
- 짧은 함수의 장점
짧은 함수는 긴 함수에 비해 이해하기가 쉽다.
- 짧은 함수의 단점
간접 호출을 하는 위임 작업이 많을 수 있다.

긴 함수는 서브 함수들로 추출할 수 있다

복잡하고 거대한 -> 작은 문제로 분해

긴 함수를 작은 서브 함수들로 분해하고 의도를 명확히 드러낸다.



추상화 계층을 신경 써서 작업하면 좋다.

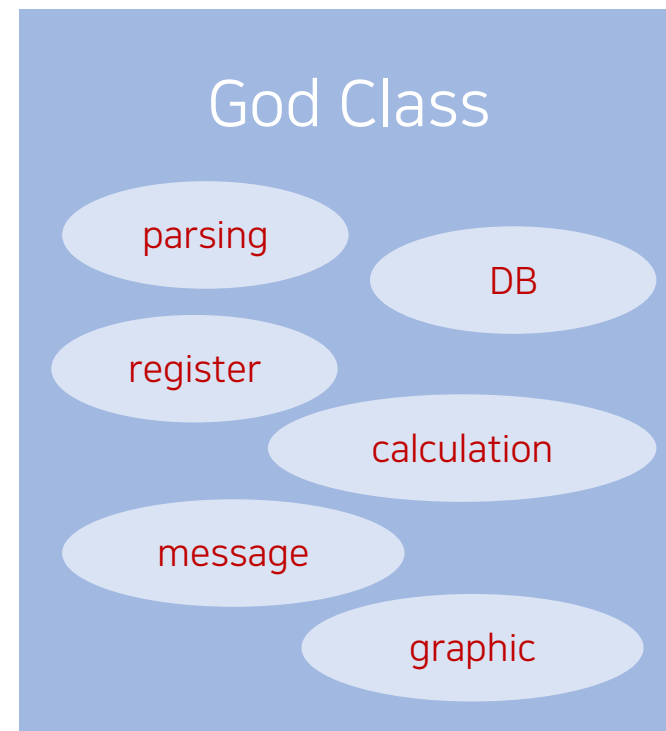
[Bad Smell] Large Class (거대한 클래스)

거대한 클래스는 너무 많은 일을 하고 있을 수 있다.

또한 긴 함수와 마찬가지로 거대한 클래스는 이해하기 어렵다.

해결 방법

- 클래스 자체에서 중복을 제거하기
- 다른 클래스로 추출하기
 - 다른 클래스로 추출 후 Composition관계
 - Super / Sub클래스로 추출



[Bad Smell] Speculative Generality (추측성 일반화)

“나중에 필요할 거야” 라는 생각으로 지금 당장 쓰지 않을 코드들을 만들어 두곤 한다. 이때 기억해야할 설계 정신은 YAGNI (You Ain't Gonna Need It) 이다.

당장 필요 없는 추상 클래스,

인터페이스와 같은 후킹(hooking)포인트를 만들어 두는 것

당장 필요 없는 매개변수

사용한 곳이 없는 함수, 클래스

위 같은 요소들에 대한 과도한 예측은 복잡성만 늘린다.

Chapter2

데이터 구성 관련 Smell

Code Smell의 분류

데이터 구성 관련 Code Smell

연관된 데이터가 흩어져 다닌다면 적절한 관리가 필요하다.

적절한 관리를 하게 된다면 유지보수성, 가독성, 확장성 등을 향상시킬 수 있다.

흩어져 있는 데이터들로 나타나는 Code Smell 패턴 종류

- 데이터 뭉치 (Data Clumps)

- 긴 매개변수 리스트 (Long Parameter List)

- 원시 타입 집착 (Primitive Obsession)

[Bad Smell] 데이터 뭉치

뭉쳐 다니는 데이터들은 묶어 줄 수(캡슐화) 있다

```
class Drawer {  
    public void drawCircle (double x, double y, double radius){}  
    public void drawTriangle (double x, double y, double height, double width){}  
    public void drawRectangle (double x, double y, double height, double width){}  
    // ...  
}
```



Point



Dimension

[Bad Smell] 긴 파라미터 리스트

생성자든 함수든 파라미터가 많으면 ...

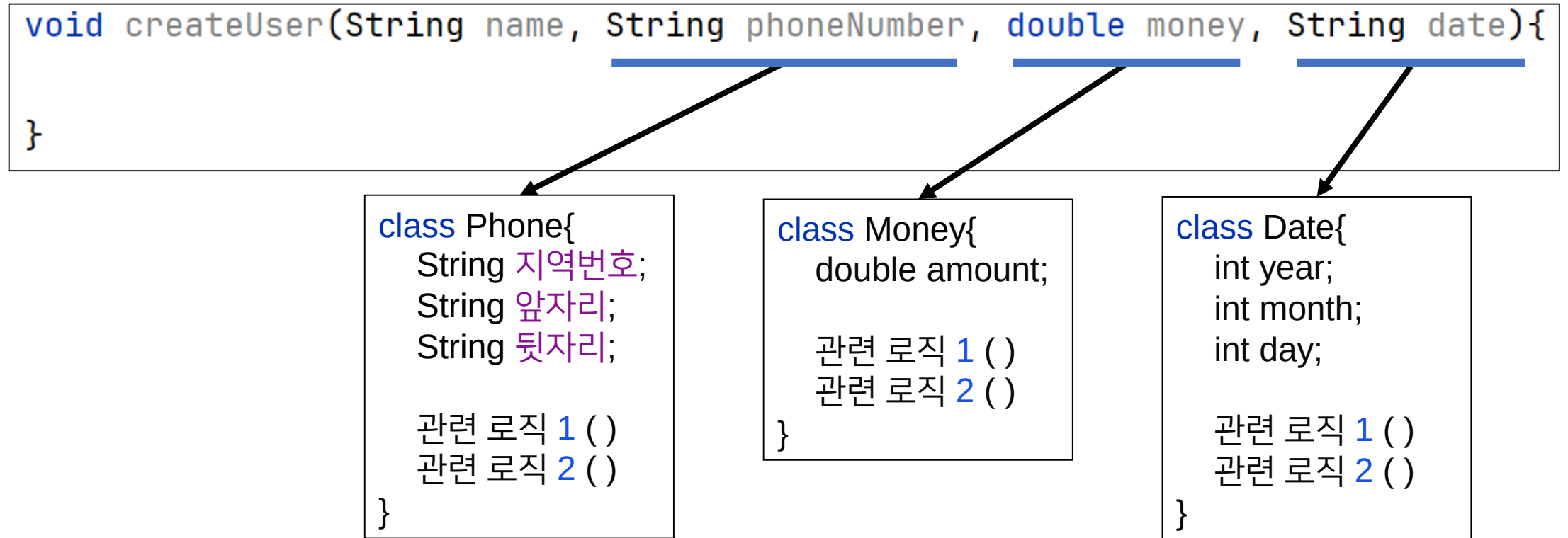
- 이해하기 어렵다.
- 테스트가 어렵다.
- 실수할 가능성이 크다.

```
Product(String name, double price, String category, int stockQuantity, String manufacturer, String supplier,  
        String sku, String color, double weight, double height, double width, double depth)
```

위와 같은 생성자를 원하는가?

[Bad Smell] 원시 타입 집착

- int, char, double, String ... 원시타입을 사용하면 표현력이 떨어지고 로직들이 흩어진다.



[리팩토링 기법] 객체 통째로 넘기기 (Preserve Whole Object)

객체로부터 값들을 얻은 후, 따로따로 넘기고 있는 경우에는 차라리 객체를 통째로 넘기도록 변경할 수 있다.

```
Point a = new Point(1.5, 2.0);  
Point b = new Point(2.5, 3.0);  
  
double dist = calculateDistance(a.getX(), a.getY(), b.getX(), b.getY());
```



```
Point a = new Point(1.5, 2.0);  
Point b = new Point(2.5, 3.0);  
  
double dist = calculateDistance(a, b);
```

Chapter3

변경 시 응집도 Smell

Code Smell의 분류

변경을 할때 응집도 저하 Code Smell

응집도가 떨어지는 코드를 변경하는 경우, 흩어져 있는 코드들로 인해서 변경 작업이 복잡해 진다.

변경하기 쉬운 코드로 만들기 위해서는 응집도를 높여야 하며, 이를 위해서 변경을 할 때 응집도가 저하된 증상을 알아 본다.

응집도가 떨어지는 증상으로 나타나는 Code Smell 패턴 종류

- 다양한 변경 이유 (Divergent Change)

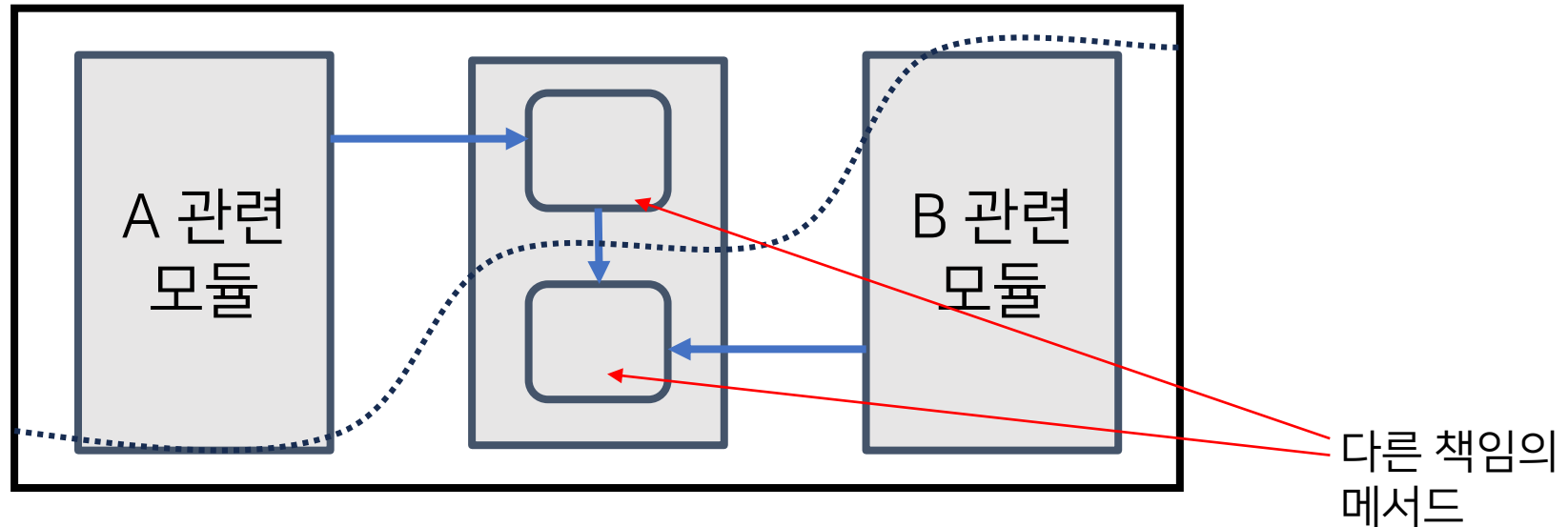
- 산탄총 수술 (Shotgun Surgery)

[Bad Smell] 서로 다른 변경 이유 (Divergent Change)

한 클래스가 다른 이유들로 자주 변경되는 경우
= 클래스가 여러 책임을 갖는 경우

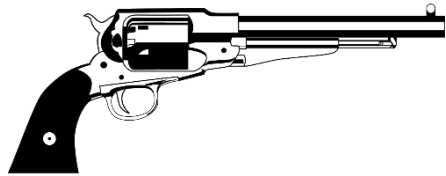
예를 들어, "새로운 데이터베이스를 추가할 때, 새로운 금융상품을 추가할 때"

이렇게 서로 다른 이유로 하나의 클래스가 변경 되는 경우
두 개의 클래스로 분리하는 작업이 필요하다.

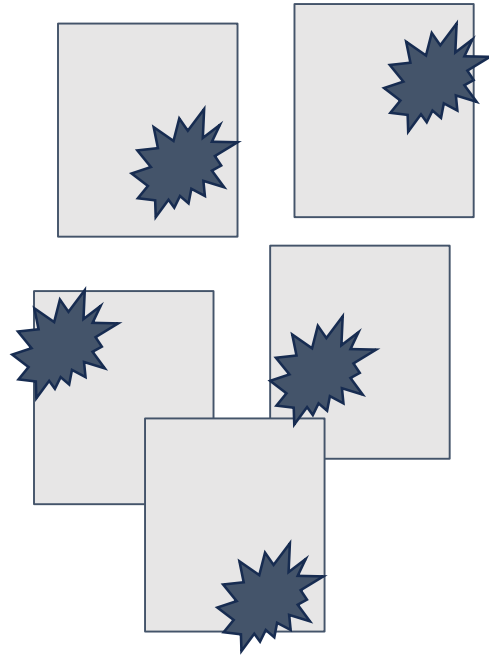


[Bad Smell] 산탄총 수술 (Shotgun Surgery)

- 특정 변경을 할 때마다 여러 클래스들을 수정해야 하는 경우
- 즉, 변경 해야하는 지점들이 여러 군데 퍼져 있는 경우이다.



변경 발생



Chapter4

객체 관계 및 의존성 문제

Code Smell의 분류

객체 관계 및 의존성 관련 Code Smell

- 다른 객체와의 의존도, 결합도가 강하다면 의존하는 클래스의 변경시 덩달아 같이 변경되어야 하는 변경에 취약한 구조가 된다. 따라서 의존도, 결합도를 높이는 요소들을 제거해야 한다.
- 의존성 관련 Code Smell 패턴 종류
 - 메시지 체인 (Message Chains)
 - 중개자 (Middle Man)

[Bad Smell] 메시지 체인

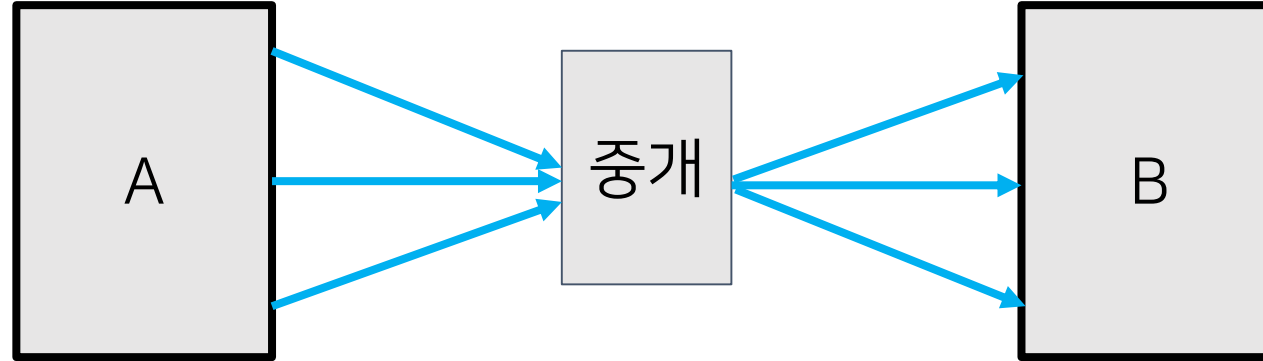
Client 가 원하는 Server 객체를 이용하려 할때, 여러 단계 걸쳐서 이용할 때 체인 구조가 나타난다.

```
Client.getOrder().getProduct().getProductName()
```

위와 같이, 메시지가 체이닝 되어있는 형태는 중간에 연결구조가 변경되면 Client 코드도 같이 변경되어야 한다.

[Bad Smell] 중개자

- 한 클래스가 하는 일 없이 다른 클래스에 구현을 위임만 하고 있다면, 중개자 역할을 해당 클래스를 제거를 하자.



중개자 클래스를 통해 B는 A로부터 숨겨져(캡슐화 되어) 있다.
이 과정에서 위임이 사용되는데, 중개자 클래스가 단순 위임만 한다면 나쁜 증상이 될 수 있다.